

UNIVERSIDADE FEDERAL DA PARAÍBA
CENTRO DE INFORMÁTICA
ENGENHARIA DE SISTEMAS DISTRIBUÍDOS

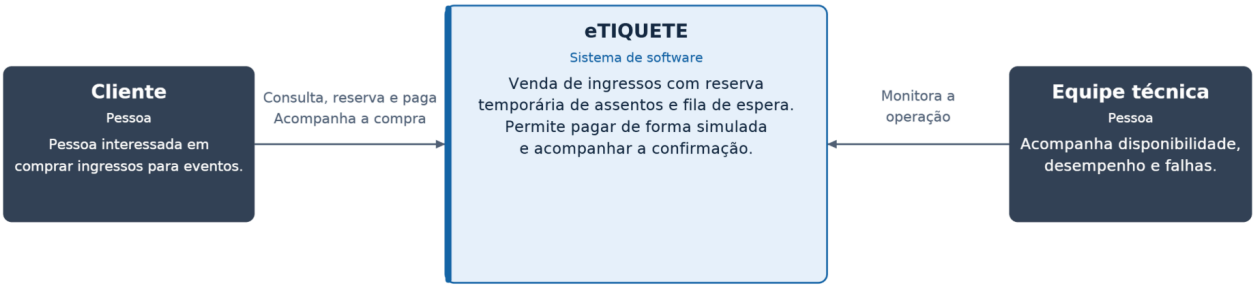
VENDA DE INGRESSOS COM RESERVA DE ASSENTOS
ETIQUETE

GRUPO

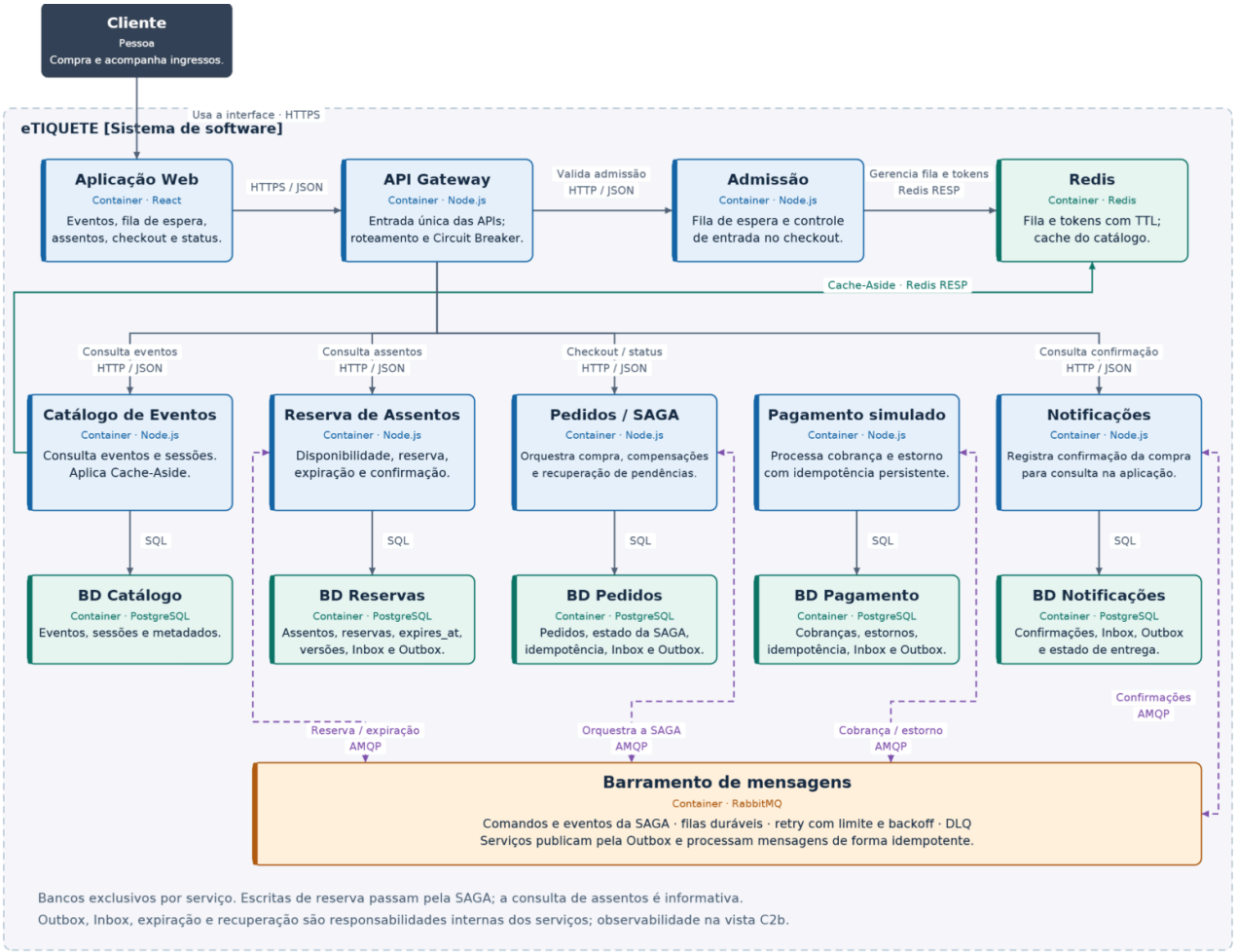
Davi de Lacerda Teixeira	(20230012251)
Enilson Lima da Costa Júnior	(20230012556)
Herick José de Freitas	(20230021813)
João Victor Fernandes da Silveira	(20230012298)
Rafael de França Silva	(20230012654)

JOÃO PESSOA
2026

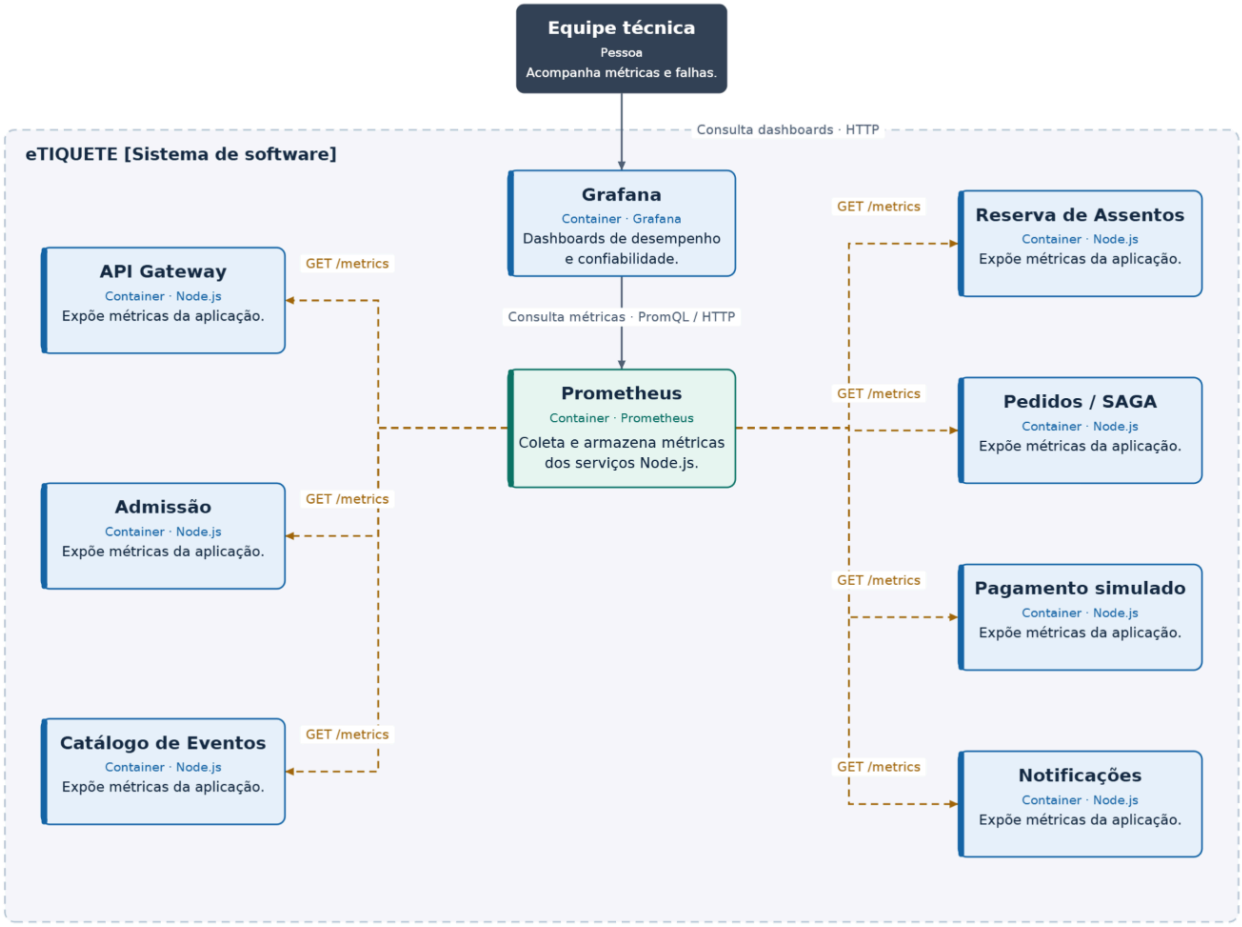
1. DIAGRAMA C4



Nível 1 – Pessoas e escopo da plataforma



Nível 2 – Vista principal da compra e coordenação distribuída



Nível 2 – Vista complementar dos mesmos contêineres de aplicação

2. RESPONSABILIDADES DOS INTEGRANTES

Integrante	Parte	Responsabilidades	Entrega prática
Herick	**Interface e API Gateway**	Frontend React, navegação entre eventos, seleção de assentos, checkout e acompanhamento da compra. Gateway com roteamento e Circuit Breaker.	O cliente consegue realizar o fluxo pela interface e visualizar espera, sucesso, expiração e falhas.
Davi	**Catálogo, cache e admissão**	Serviço de eventos, PostgreSQL do catálogo, Cache-Aside e Redis. Fila de espera, tokens temporários e limite de entrada no checkout.	Eventos podem ser consultados com cache, e o sistema controla quantos usuários entram na compra simultaneamente.
João Victor	**Reservas e concorrência**	Inventário de assentos, banco de reservas, aquisição atômica, confirmação, liberação e rotina de expiração.	Duas compras concorrentes não conseguem reservar o mesmo assento; reservas abandonadas são liberadas.
Rafael	**Pedidos e coordenação distribuída**	Serviço de Pedidos, orquestração da SAGA, RabbitMQ, contratos de mensagens, retry, DLQ e recuperação de pedidos incompletos.	A compra avança entre os serviços e executa compensações quando alguma etapa falha.
Enilson	**Pagamentos e confirmações**	Pagamento simulado, idempotência de cobrança e estorno, serviço de Notificações e persistência das confirmações.	Repetir uma solicitação não duplica a cobrança; compras concluídas recebem confirmação e pagamentos tardios podem ser estornados.

3. DECISÕES ARQUITETURAIS

Código	Nome	Status	Contexto	Decisão	Consequências
ADR 0001	Microsserviços com banco exclusivo	Aceita	A venda de ingressos envolve catálogo, reserva, pagamento e notificação. Um banco compartilhado faria cada mudança de módulo arrastar os outros.	Cada microsserviço persiste no próprio banco ou esquema. Os demais só acessam esses dados por API ou mensagem.	Catálogo, reserva, pedido, pagamento e notificação evoluem separados. Uma compra não cabe numa transação única entre todos os bancos.
ADR 0002	SAGA orquestrada com RabbitMQ	Aceita	A compra passa por reserva, pagamento e confirmação. Falha no pagamento ou reserva expirada precisa desfazer o que já ocorreu, sem transação distribuída entre os serviços.	O serviço de Pedidos orquestra a SAGA. O RabbitMQ leva as mensagens entre pedidos, reservas, pagamento e notificações, com fila durável, confirmação, retry e DLQ. Se uma etapa falha ou a reserva expira, o orquestrador pede a compensação: libera os assentos e, se o pagamento já ocorreu, estorna a cobrança.	O pedido incompleto se recupera por compensação. Retry trata falha passageira; a DLQ guarda a mensagem que esgotou as tentativas.

4. STACK TECNOLÓGICO

Tecnologia	Justificativa de uso no projeto
NodeJS + React	Os microsserviços rodam em Node.js, que trata I/O assíncrono na conversa com banco, API e fila. O React monta a interface do catálogo, do mapa de assentos, da fila e do acompanhamento da compra, com componentes reutilizáveis.
PostgreSQL	Guarda eventos, reservas, pedidos e pagamentos. Transação e bloqueio impedem duas compras de levarem o mesmo assento. No mesmo banco ficam o estado da SAGA, as chaves de idempotência e a Outbox, com banco ou esquema exclusivo por serviço.
Redis	Serve de cache do catálogo, para não repetir a mesma consulta no PostgreSQL, e guarda os tokens de admissão com prazo. A expiração de chave cobre esse prazo. Dono e validade da reserva continuam no PostgreSQL.
RabbitMQ	Envia as mensagens da SAGA entre pedidos, reservas, pagamento e notificações. Fila durável e confirmação de publicação e de processamento sustentam a entrega. Retry e DLQ separam falha passageira de mensagem que precisa de análise ou reprocessamento.
Docker Compose	Sobe os microsserviços e as dependências na máquina local, com rede, volume e configuração. O grupo integra as partes e reproduz o ambiente para teste e demonstração.
GitHub Actions	Roda verificação de código, teste e build a cada alteração e em pull request. A falha aparece antes de juntar os serviços. Publicação e implantação podem entrar nesse fluxo depois.
Testcontainers	Sobe PostgreSQL, Redis e RabbitMQ descartáveis no teste de integração. Dá para ver transação, expiração e mensageria perto da execução real, inclusive concorrência e reprocessamento.

Prometheus	Coleta e guarda métricas dos serviços: latência, taxa de erro, conflito de reserva e duração da compra. Com elas o grupo confere confiabilidade e desempenho e localiza gargalo no teste de carga.
Grafana	Mostra as métricas do Prometheus em dashboards. Os painéis cruzam volume de acesso, tempo de espera, falha de pagamento e compensação, para ler o resultado do teste e investigar o problema.

5. PADRÕES ARQUITETURAIS

Padrão arquitetural	Descrição básica
SAGA	Organiza uma operação distribuída em transações locais executadas por diferentes serviços. Quando uma etapa falha, ações compensatórias desfazem os efeitos das etapas anteriores, como liberar uma reserva ou estornar um pagamento.
Transactional Outbox	Registra a alteração dos dados e a mensagem a ser publicada na mesma transação do banco. Depois, um processo publica as mensagens pendentes, evitando que uma alteração seja confirmada sem o correspondente registro para envio.
Retry e DLQ	**Retry** repete a operação que falhou por um tempo, com intervalo e limite de tentativas. A **DLQ** recebe a mensagem que não concluiu o processamento, para análise e reprocessamento, sem travar o fluxo principal.
Circuit Breaker	Interrompe temporariamente chamadas a um serviço que apresenta falhas recorrentes, evitando sobrecarga e propagação de problemas. Após um período, permite chamadas de teste para verificar se o serviço se recuperou.
API Gateway	É o ponto de entrada dos clientes. Recebe a requisição e encaminha ao serviço responsável. Pode concentrar autenticação, limite de requisição e monitoramento.
Cache-Aside	A aplicação consulta primeiro o cache. Se o dado não estiver disponível, busca no banco e adiciona o resultado ao cache para acelerar consultas futuras. Quando o dado é alterado, a entrada correspondente deve ser invalidada ou atualizada.
Database-per-service	Cada microsserviço guarda os próprios dados num banco ou esquema exclusivo. Os outros serviços só chegam nesses dados por API ou mensagem.

REPOSITÓRIO: https://github.com/DavideLacerdaT/Etiquete